

Deep Learning Practical 4

Improving architectures

1 Overview

Welcome to the fourth deep learning practical. We will improve the encoder–decoder from Practical 3 using tensor operations, residual connections and nested blocks. In this practical, we will be covering:

- Tensor manipulation with Einstein operations.
- Convolutional architecture design.
- Residual architectures.
- Block design.

If you complete this practical early, please advance to the next practical.

For further examples of composing layers and blocks in PyTorch and JAX, see the D2L builders' guide.

2 Tensor manipulation with Einstein operations

A running joke in our lab a few years ago was that deep learning research involves 90% tensor wrangling and 10% actual research. In other words it used to be very difficult getting your input data processed and arranged in the right memory layout, such that it's suitable for feeding into a neural network.

Inspired by Einstein notation, 'einops' makes the process of reshaping and manipulating tensors much easier, more optimised, and less error-prone.

Let's start with a colour image of some coffee, where we want to simulate a batch of data by repeating it 10 times across the batch dimension:

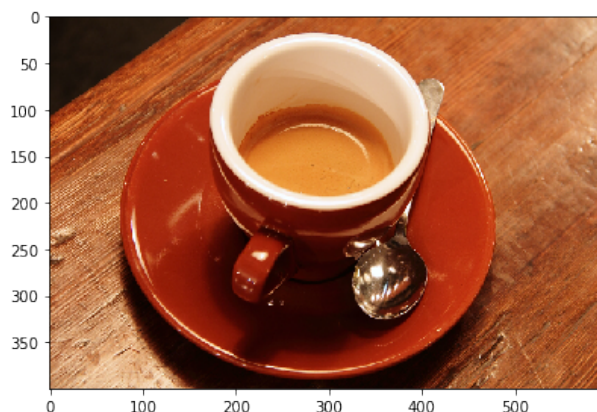
```
!pip install einops
!pip install scikit-image </> copy code

import skimage
import torch
import einops

from skimage import data, io
image = data.coffee()
io.imshow(image)

x = torch.tensor(image)
x = x.float()/x.max()
print(x.size())

> torch.Size([400, 600, 3])
```



Oh no—this is no good! This is $H \times W \times C$ (height, width, channel) but we want $B \times C \times H \times W$, and we also want to repeat it 10 times across the batch dimension. While cumbersome and sometimes inefficient to permute the tensor to the desired memory layout manually in PyTorch, it is very easy to achieve this with einops:

```
from einops import rearrange, reduce, repeat

coffee_batch = repeat(x, 'h w c -> b c h w', b=10)
print(coffee_batch.size())
```

- Run the above code and confirm it has worked as intended.

Einops supports built-in layers for all mainstream deep learning libraries (torch, tensorflow, JAX via flax etc):

```
from einops.layers.torch import Rearrange, Reduce

nn.Sequential(
    Rearrange(...),
    nn.Linear(...),
    Reduce(...)
)
```

Exercise

- Create a new multilayer perceptron MLP(`nn.Module`) class. This will just be a simple network with two `nn.Linear(...)` layers.
- Given your `coffee_batch` tensor of $B \times C \times H \times W$, and using the einops layer ‘Rearrange’, construct a 2-layer MLP whose first layer has $3 \times 400 \times 600$ input features (for each colour pixel) and 5 outputs.
 - In einops, you can flatten dimensions by putting them in circular brackets.
 - For example ‘`b c h w -> b (c h w)`’ will keep the batch dimensions and flatten the channels and spatial dimensions into a vector, making it suitable for input to a linear layer.
- Use a ReLU nonlinearity and make the second linear layer map from 5 inputs to 1 output. Put all these layers in a `self.layers = nn.Sequential(...)` and return a call to this in the forward pass.
- Initialise `net = MLP()`
- Evaluate `net(coffee_batch).size()` and confirm the output is `torch.Size([10, 1])`.
- Have a look through <http://einops.rocks/pytorch-examples.html> and study the different einops uses cases of ‘rearrange’ and ‘reduce’.
- You can do very complicated manipulations easily. Run the following and explain what is happening:

```
img_grid = rearrange(coffee_batch, '(b1 b2) c h w -> (b1 h) (b2 w) c', b1=2)
io.imshow((img_grid*255).int().numpy())
```

3 Convolutional neural networks

A convolution is a sliding dot product between a matrix (kernel) and the section of the signal (image, audio...) that it’s aligned with. The kernel is typically something small, like $4 \times 4 = 16$ parameters, and it remains the same size regardless of how big or small the input image is. In contrast, a linear layer requires as many parameters as there are pixel inputs in the image, multiplied by the number of desired output neurons. In other words, a linear layer requires *a lot* of parameters for large images, text sequences, or audio clips.

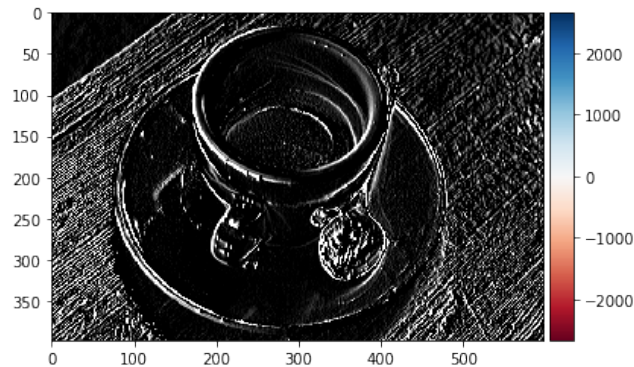
A convolutional output channel sums contributions from all input channels. For an RGB image, its 4×4 kernel therefore contains $3 \times 4 \times 4 = 48$ weights, with one additional parameter if a bias is used. To see how a fixed filter operates before learning any weights, apply this 3×3 Sobel filter to each channel and sum the responses:

```
conv = nn.Conv2d(in_channels=3, out_channels=1, kernel_size=3, bias=False)
print(conv.weight.size())
print('Number of parameters: ' + str(len(torch.nn.utils.parameters_to_vector(conv.parameters()))))
print(conv.weight.requires_grad)

w = repeat(torch.tensor([
    [-1,0.,1],
    [-2,0.,2],
    [-1,0.,1]
]), 'h w -> out_f in_f h w', out_f=1, in_f=3)

conv.weight.data = w
out = conv(coffee_batch)

io.imshow((repeat(out[0,0], 'h w -> h w c', c=3)*255).int().numpy())
```



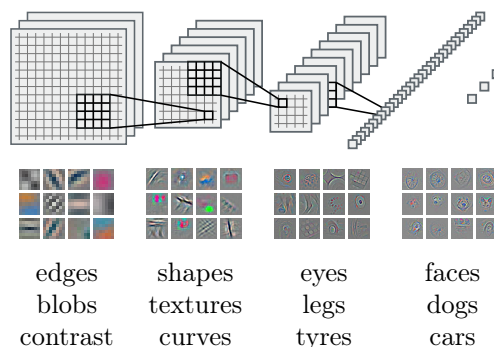
The main difference in deep learning is that we wish to learn these weights, so they're initialised randomly and have `requires_grad=True` set by default. Also, just like we need to convolve separately for each input channel, we also need to convolve separately for each output channel (detector) that we wish to learn. For example, the Sobel filter example above only detects vertical edges in one orientation, but if we wish to learn to detect 24 different types of feature (e.g. horizontal edges, blobs, textures, corners, ...), we would need to learn 24 separate $3 \times 4 \times 4$ kernels, so our final weights tensor would be rank 4 with $24 \times 3 \times 4 \times 4 = 1152$ parameters for a 4×4 kernel. But that's still far fewer than the 3.6 million parameters we required for our linear layer earlier.

Convolutional design

So how many output channels? Well, generally there are two things we wish to achieve: encoding and decoding. When we encode, we wish to learn a latent representation (feature vector) that describes the image abstractly. So the image may be represented by a feature vector with a small compressed set of neurons that activate if the image contains faces, red hair, trees, cars etc.

To form a latent encoding, we often reduce the spatial dimensions while increasing the number of channels. Halving both height and width leaves one quarter as many spatial positions:

Encoder input: $B \times 3 \times 128 \times 128$
Middle layer: $B \times 64 \times 8 \times 8$
Encoder output: $B \times 512 \times 1 \times 1$



Similarly, we can decode a latent representation back to something of the same shape as the original input, e.g.:

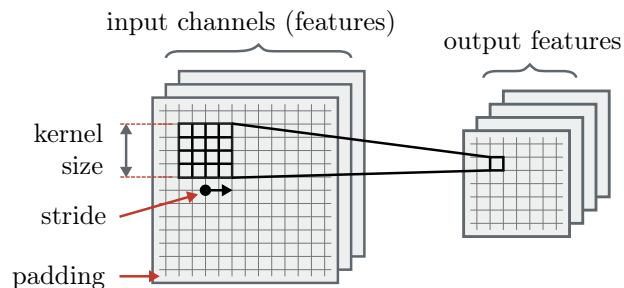
Decoder input: $B \times 512 \times 1 \times 1$
Middle layer: $B \times 64 \times 8 \times 8$
Decoder output: $B \times 3 \times 128 \times 128$

How quickly should we reduce the spatial dimensions and increase the channels? This depends on the dataset and task: downsampling lets later features combine information over larger regions, but can discard fine spatial detail. Tasks such as segmentation need that detail in the final output, which motivates preserving earlier features through skip connections.

How do we reduce the spatial dimensions? Either use max or mean pooling layers, equivalent to this einops:

```
reduce(x, 'b c (h1 h2) (w1 w2) -> b c h1 w1', 'max', h2=2, w2=2)
```

Or we can change the stride and padding of our convolutional layers. These are best illustrated:



Stride controls the spacing between successive kernel positions. With kernel size 4, stride 2 and padding 1, `nn.Conv2d` halves each even spatial dimension, rounding down for odd sizes. A matching `nn.ConvTranspose2d(..., kernel_size=4, stride=2, padding=1)` doubles each spatial dimension, which is useful in decoders.

- To gain some further intuition of convolution and transpose convolution, visit https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md and look at the visualisations.
 - Blue are the inputs (images) and green are the outputs.

Two common convolution patterns

For simplicity and excellent results in the majority of convolutional architectural designs, you can stick to the following two common patterns of kernel size, stride, and padding accordingly:

- The 3,1,1 pattern (kernel size = 3, stride = 1, padding = 1).
 - Allows you to change output channels to anything, without changing the spatial dimension(s).
- The 4,2,1 pattern.
 - Halves each spatial dimension, rounding down for odd sizes.

A 1×1 convolution with stride 1 and padding 0 applies the same affine map to the channels at every spatial position. With a 1×1 input, it is equivalent to a linear layer. When upsampling, the choice of kernel size and stride can produce checkerboard artefacts; explore the interactive examples at <https://distill.pub/2016/deconv-checkerboard/>.

Exercise

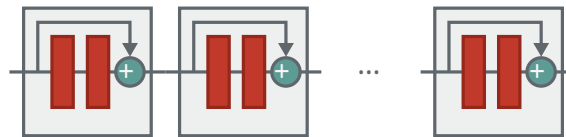
1. Design a 5 layer convolutional encoder that takes as input a rank 4 tensor of shape $B \times 3 \times 32 \times 32$ and outputs a representation of shape $B \times 1024 \times 1 \times 1$. Do not use max or mean pooling.
 - Start with `x0 = torch.zeros(10, 3, 32, 32)`

- `c1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=4, stride=2, padding=1)`
`x1 = c1(x0)`
`print(x1.size())`
 - `c2 = nn.Conv2d(...)`
`x2 = c2(x1)`
`print(x2.size())`
`...`
2. After you’ve designed the network, make an ‘Encoder’ class and aggregate the layers in `nn.Sequential(...)`. Feed `x0` into the network, and confirm it outputs a tensor with `torch.Size([10, 1024, 1, 1])`.
 3. How many parameters does each layer in your architecture use?
 4. How would you design an architecture that has fewer parameters, but still outputs a 1024-dimensional latent representation?
 5. How would you adjust the architecture for non-square input images of size $B \times 3 \times 32 \times 64$? Hint: kernel size, stride and padding can each be a pair of integers, one for each spatial axis.
 6. Design a 5 layer convolutional decoder that transforms the $B \times 1024 \times 1 \times 1$ latent representation back to $B \times 3 \times 32 \times 32$. Use `nn.ConvTranspose2d(...)` for this.

4 Residual architectures

Adding layers increases the number of transformations that a network can represent, but can also make optimisation harder. Backpropagation multiplies layer Jacobians, which can cause gradients to shrink or grow. Batch normalisation changes the scale of intermediate activations; its effect depends on the architecture and training procedure.

A residual connection adds an input to a learned transformation: $y = x + F(x)$. Its Jacobian is $I + J_F(x)$, so gradients have a direct identity contribution. The tensors being added must have the same shape. The example below uses convolutions with kernel size 3, stride 1 and padding 1 to preserve spatial dimensions, followed by a ReLU after the addition.



```
class ResidualBlock(nn.Module):
    def __init__(self, n):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(n,n,3,1,1),
            nn.BatchNorm2d(n),
            nn.ReLU(),
            nn.Conv2d(n,n,3,1,1),
            nn.BatchNorm2d(n)
        )
    def forward(self, x):
        return torch.relu(x + self.block(x))
```

[copy code](#)

These can then be inserted as with any other layer, for example as part of a `nn.Sequential` block—you just need to specify the number of input features.

5 Block design

Exercise

- Add `ResidualBlock` layers to the `nn.Sequential` part of your `Encoder` and `Decoder` classes. Check the networks still work as intended and print the number of parameters.
- Create an `Autoencoder` class that initialises `self.encoder = Encoder()` and `self.decoder = Decoder()`. Evaluate your autoencoder on `x0` and confirm it returns a tensor of the same shape.

- This approach of designing and assembling deep neural networks hierarchically from blocks can be powerful and intuitive. The majority of state-of-the-art architectures introduce new blocks and then assemble them sequentially or hierarchically to create very deep and expressive high-capacity models.

Use these blocks to improve the colourisation model from Practical 3. Compare it with the same training split, batch size and update count. Practical 5 uses these architectural ideas to build a generator and discriminator.

6 Improve the CIFAR-10 colouriser

Building on the MLP colouriser in Practical 3, retain its direct greyscale path and replace the fully connected layers with convolutional stages. You can then investigate an additional skip connection carrying the encoder's 16×16 feature maps to the decoder. Work through `dl-practical4.py`, which includes the data loading, training and plotting code.

A block inside a stage inside a network

```
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.layers=nn.Sequential(nn.Conv2d(channels,channels,3,1,1),nn.ReLU(),
                                   nn.Conv2d(channels,channels,3,1,1))

    def forward(self,x):
        return F.relu(x+self.layers(x))

class Stage(nn.Module):
    def __init__(self,inputs,outputs):
        super().__init__()
        self.layers=nn.Sequential(nn.Conv2d(inputs,outputs,4,2,1),nn.ReLU(),
                                   ResidualBlock(outputs),ResidualBlock(outputs))

    def forward(self,x):return self.layers(x)
```

Each stage halves the spatial resolution, then applies two residual blocks. Combine two stages in the encoder and upsample their features in the decoder:

```
class Colouriser(nn.Module):
    def __init__(self, feature_skip=False):
        super().__init__()
        self.feature_skip=feature_skip
        self.encoder=nn.Sequential(Stage(1,16),Stage(16,32))
        self.up=nn.Sequential(nn.Upsample(scale_factor=2,mode='nearest'),
                               nn.Conv2d(32,16,3,1,1))
        self.decoder=nn.Sequential(ResidualBlock(16),nn.Upsample(scale_factor=2,mode='nearest'),
                                   nn.Conv2d(16,3,3,1,1))
        nn.init.zeros_(self.decoder[-1].weight)
        nn.init.zeros_(self.decoder[-1].bias)

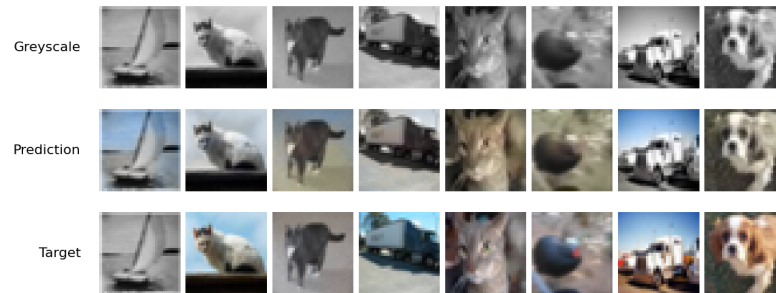
    def forward(self,x):
        h=self.encoder[0](x)
        z=self.encoder[1](h)
        up=self.up(z)
        if self.feature_skip:up=up+h
        return x.expand(-1,3,-1,-1)+self.decoder(up)
```

Compare three forms of reuse: repeated layers inside a residual block, repeated residual blocks inside a stage, and two stages inside the encoder. Print the network and match each level to its part of the computation. The first stage returns $h \in \mathbb{R}^{B \times 16 \times 16 \times 16}$; the second returns $z \in \mathbb{R}^{B \times 32 \times 8 \times 8}$. Upsampling z gives a tensor with the same shape as h . The decoder uses $U(z)$ by default, or $U(z) + h$ when `feature_skip=True`. Its final colour correction is added to the greyscale input. The zero-initialised output layer starts this network at the repeated-grey baseline. The helper clips predictions for display and evaluation.

Exercise

1. Check $f(x)$ and x have equal shapes at each residual addition. Explain why adding them changes the Jacobian from J_f to $I + J_f$ before the final activation.
2. Replace one stage by a stage with no residual blocks. Train for the same 6,000 updates as Practical 3 and compare validation MSE, parameter count and images.
3. Train with `feature_skip=False`, then with `feature_skip=True`. The parameter count is unchanged. Compare the effect of passing the intermediate features to the decoder.
4. Remove the direct greyscale path. Does the model spend more capacity reproducing luminance? Compare both reconstructions on the same eight examples.

5. Compare one wider stage with two narrower residual blocks at a similar parameter count. Keep the data, seed, batch and updates fixed.
6. Use `reduce(error, 'b c h w -> c', 'mean')` to examine per-channel error. Use `rearrange` to show input, output and target rows in a single image.



These are the eight validation images from Practical 3. With 64,467 parameters, the convolutional model achieves validation RGB MSE 0.005028 after 6,000 updates, batch size 64 and learning rate 0.001. The 64-value skip MLP has 1,084,992 parameters and MSE 0.005799 with these settings. Compare sky, vegetation, fur and edges; both models average over uncertain colours.

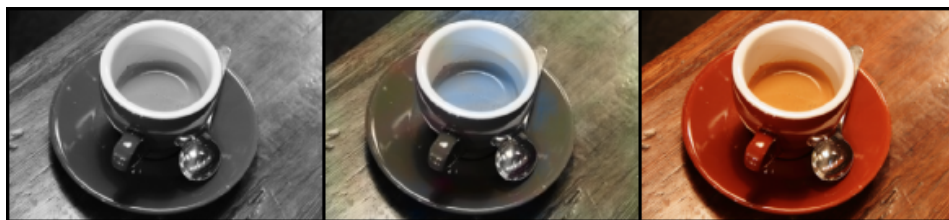
Does more training improve the colours?

Compare 1,500, 6,000 and 15,000 updates, inspecting the same photographs and plotting training and validation error. In a run at learning rate 0.002, validation MSE was 0.005264, 0.005164 and 0.005218, while training error continued to fall. More training did not improve validation error consistently. Start with the default learning rate of 0.001, varying one setting at a time.

7 Apply the convolutional network to coffee

To apply the network to the larger coffee photograph, resize it to 128×192 , convert it to grey, and run the trained convolutional model without fitting any new weights.

```
photo = F.interpolate(coffee_batch[:1], (128, 192), mode="area").to(device)
with torch.no_grad():
    prediction = model(grey(photo))
print(photo.shape, prediction.shape)
```



Exercise

Why can the convolutional network accept this resolution, while the coffee MLP needs a fixed input size? Try dimensions that are not multiples of four and diagnose any mismatch. Does the grey input determine the coffee colour uniquely? How would you assess colour accuracy here?